

Package ‘dataplane’

February 12, 2026

Title Data Architecture and Processing Helpers
Version 1.0.0
Description Tools for structuring, validating, and transforming datasets with a focus on repeat-able pipelines and clear metadata.
License MIT + file LICENSE
Encoding UTF-8
RoxygenNote 7.3.3
URL <https://jclark50.github.io/dataplane/>, <https://github.com/jclark50/dataplane>
BugReports <https://github.com/jclark50/dataplane/issues>
Imports weathertools,
 arrow,
 data.table,
 digest,
 dplyr,
 httr2,
 jsonlite,
 aws.s3,
 lubridate,
 Rcpp
LinkingTo Rcpp
Suggests knitr,
 rmarkdown,
 crayon,
 sf,
 doParallel
VignetteBuilder knitr

Contents

cbind_list	3
check_ram	3
collapse	4
dp_check_units	5
dp_detect	6
dp_open	6

dp_print_detect	7
dp_read	8
dp_read_dataset_meta	8
dp_read_meta	9
dp_scale_decode	10
dp_scale_encode	11
dp_set_scale	12
dp_set_units	12
dp_spec	13
dp_spec_default	14
dp_tag_units	15
dp_write	16
dp_write_dataset	17
dp_write_dataset_meta	18
drop_null	19
env_close	20
env_start	21
filter_cols	22
flatten_list_columns	23
gp_geoparquet_attach_geo_to_table	23
gp_geoparquet_build_geo_meta	24
gp_geoparquet_geo_json	25
gp_geoparquet_infer_from_sf	25
gp_geoparquet_is_geoparquet	26
gp_geoparquet_read_geo	27
gp_geoparquet_write_parquet	27
mavtime	28
merge_dt_list	29
null_coalesce	29
print.dp_spec	30
rep_var	30
require_ram	31
round_cols	31
s3_create_dir	32
s3_download_file	33
s3_list_objects	33
s3_upload_file	35
sanitize_value	36
stop_cluster	37
syn_create_file_entity	37
syn_create_folder	38
syn_download_file	39
syn_download_file_path	40
syn_ensure_folder_path	41
syn_get_annotations	41
syn_get_child_by_name	42
syn_get_entity	43
syn_lookup_child_id	43
syn_request	44
syn_resolve_entity_by_path	45
syn_resolve_token	46
syn_set_annotations	47

Arguments

threshold	Numeric. Minimum amount of free memory required.
unit	Character. Unit for the threshold: one of "B", "KB", "MB", or "GB" (default "MB").
wait_time	Numeric. Seconds to wait between retries (default 60).
max_attempts	Integer. Maximum number of attempts before quitting the R session with status 1 (default 3).

Details

On Windows, uses 'wmic OS get FreePhysicalMemory' to retrieve free RAM. On Unix-like systems, calls 'free -b' and parses the "available" column.

Value

Returns 'TRUE' if available RAM meets or exceeds the threshold. Otherwise, the session is terminated via 'quit(status = 1)'.

Examples

```
## Not run:
# Check for at least 2 GB of free memory, retrying up to 5 times
check_ram(threshold = 2000, unit = "MB", wait_time = 30, max_attempts = 5)

## End(Not run)
```

collapse

*Collapse strings into a quoted, delimited single string***Description**

Convenience helper for building strings like: 'a', 'b', 'c'

Usage

```
collapse(someStrings, collapsewith = "'", '"', leadendwith = "'")
```

Arguments

someStrings	Vector/list of strings.
collapsewith	Character. Delimiter placed between values.
leadendwith	Character. String placed at the start and end (often a quote).

Value

A single character string.

Examples

```
collapse(c("a", "b", "c"))
collapse(c("x", "y"), collapsewith = ",", leadendwith = "\"")
```

dp_check_units	<i>Check unit attributes against a spec</i>
----------------	---

Description

Compares unit attributes in a table (default attribute name "units") against expected units from a 'dp_spec'. By default, this warns and returns a report.

Usage

```
dp_check_units(
  dt,
  spec,
  system = c("declared", "metric", "imperial"),
  unit_attr = "units",
  warn_max = 8L,
  stop_on_any = FALSE
)
```

Arguments

dt	A 'data.frame' or 'data.table'.
spec	A 'dp_spec' with a 'fields' table.
system	Which unit system to validate against: "declared" (default), "metric", or "imperial".
unit_attr	Attribute name to read (default "units").
warn_max	Maximum number of problems to show in the warning message.
stop_on_any	Logical. If 'TRUE', stop if any problems are found.

Value

An audit 'data.table' (invisibly) describing expected vs observed units and a status code.

Examples

```
## Not run:
library(data.table)
dt <- data.table(temp = 25.1)
sp <- dp_spec_default(dt, "metric")
dt2 <- dp_tag_units(dt, sp)$dt
dp_check_units(dt2, sp)

## End(Not run)
```

dp_detect	<i>Detect whether a file or dataset contains Dataplane metadata</i>
-----------	---

Description

Works on a single Parquet file or a dataset directory. For dataset directories, this function prefers a Dataplane sidecar (e.g., 'dp_meta.parquet') and otherwise samples Parquet files and applies a majority rule.

Usage

```
dp_detect(
  path,
  preferred_prefix = "dp",
  mode = c("detect", "validate"),
  scan_n = 5L,
  recursive = TRUE
)
```

Arguments

path	Path to a Parquet file or dataset directory.
preferred_prefix	Preferred metadata prefix (default "dp").
mode	"detect" or "validate". "validate" uses a stricter threshold.
scan_n	For dataset directories without a sidecar, number of Parquet files to sample.
recursive	For dataset directories, whether to scan recursively for Parquet files.

Value

A list describing detection outcome (confidence, evidence keys, etc.).

Examples

```
## Not run:
dp_detect("file.parquet")
dp_detect("dataset_dir")

## End(Not run)
```

dp_open	<i>Open an Arrow Dataset and (optionally) read Dataplane sidecar metadata</i>
---------	---

Description

Opens a dataset directory via [arrow::open_dataset()] and optionally reads a sidecar metadata file (e.g., 'dp_meta.parquet').

Usage

```
dp_open(
  path,
  metadata = c("sidecar", "none"),
  ignore_prefixes = c("_dp_"),
  exclude_invalid_files = TRUE,
  preferred_prefix = "dp",
  ...
)
```

Arguments

path	Dataset directory path.
metadata	"sidecar" or "none". If "sidecar", also read dataset metadata.
ignore_prefixes	File prefixes to ignore when opening dataset (default includes '_dp_').
exclude_invalid_files	Passed to Arrow dataset factory options.
preferred_prefix	Preferred metadata prefix (default "dp").
...	Passed to [arrow::open_dataset()].

Value

If 'metadata="none"', an Arrow Dataset. If 'metadata="sidecar"', a list: - 'ds': Arrow Dataset - 'meta': parsed sidecar metadata

dp_print_detect	<i>Print a Dataplane detection result</i>
-----------------	---

Description

Convenience printer for objects returned by [dp_detect()].

Usage

```
dp_print_detect(x)
```

Arguments

x	Detection result list from [dp_detect()].
---	---

Value

The input 'x' (invisibly).

dp_read

*Read a Parquet file written with Dataplane metadata***Description**

Reads a Parquet file and returns both the data and parsed Dataplane metadata. Optional behaviors:

- decode scaled integer storage columns back to numeric columns
- attach declared unit attributes using the stored spec

Usage

```
dp_read(
  path,
  decode_scaled = TRUE,
  keep_storage = TRUE,
  attach_units = c("declared", "none"),
  preferred_prefix = "dp",
  unit_attr = "units",
  ...
)
```

Arguments

path	Path to a Parquet file.
decode_scaled	Logical. If 'TRUE', decode scaled columns using the audit table.
keep_storage	Logical. If 'TRUE', keep storage columns after decoding.
attach_units	"declared" or "none". If "declared", attach unit attributes based on the stored spec.
preferred_prefix	Preferred metadata prefix (default "dp").
unit_attr	Unit attribute name (default "units").
...	Passed to [arrow::read_parquet()].

Value

A list with: - 'dt': a 'data.table' - 'meta': parsed metadata from [dp_read_meta()]

dp_read_dataset_meta

*Read Dataplane dataset metadata sidecar***Description**

Reads a dataset directory for a known sidecar metadata file (e.g., 'dp_meta.parquet') and returns parsed Dataplane metadata.

Usage

```
dp_read_dataset_meta(
  dataset_path,
  parse = TRUE,
  preferred_prefix = "dp",
  sidecar_names = NULL
)
```

Arguments

`dataset_path` Dataset directory path.

`parse` Logical. If 'TRUE', parse serialized payloads into 'spec_dt'/'audit_dt'.

`preferred_prefix` Preferred metadata prefix (default "dp").

`sidecar_names` Optional vector of sidecar names to look for.

Value

Same structure as `[dp_read_meta()]`, plus 'sidecar_path'.

<code>dp_read_meta</code>	<i>Read Dataplane metadata from a Parquet file (metadata-only)</i>
---------------------------	--

Description

Reads schema key/value metadata from a Parquet file and (optionally) parses the Dataplane payloads ('spec_dt', 'audit_dt').

Usage

```
dp_read_meta(path, parse = TRUE, preferred_prefix = "dp")
```

Arguments

`path` Path to a Parquet file.

`parse` Logical. If 'TRUE', parse serialized payloads into 'spec_dt' and 'audit_dt'.

`preferred_prefix` Preferred metadata prefix (default "dp").

Value

A list with: - 'prefix': detected prefix - 'keys': resolved key names for the detected prefix - 'kv': raw key/value list - 'spec_dt': parsed spec table (or 'NULL') - 'audit_dt': parsed audit table (or 'NULL')

Examples

```
## Not run:
m <- dp_read_meta("file.parquet")
names(m$kv)

## End(Not run)
```

dp_scale_decode	<i>Decode scaled integer columns back to numeric columns (using audit metadata)</i>
-----------------	---

Description

If a writer created columns like 'temp_i' with 'writer_scale = 100', this can recreate 'temp = temp_i / 100'. Optionally keeps or drops the integer storage columns.

Usage

```
dp_scale_decode(dt, audit_dt, keep_storage = TRUE, overwrite = FALSE)
```

Arguments

dt	A 'data.frame' or 'data.table'.
audit_dt	Parsed audit/plan table. Must contain: 'column', 'writer_scale', 'storage_column', 'did_scale'.
keep_storage	Logical. If 'TRUE', keep the integer storage columns.
overwrite	Logical. If 'TRUE', overwrite existing base columns if present.

Details

'audit_dt' is typically obtained from metadata via [dp_read_meta()] or from the 'meta' element returned by [dp_read()].

Value

A decoded 'data.table' (modified copy).

Examples

```
## Not run:
library(data.table)
dt <- data.table(temp_i = c(2512L, 2544L))
audit <- data.table(column="temp", writer_scale=100, storage_column="temp_i", did_scale=TRUE)
dp_scale_decode(dt, audit)

## End(Not run)
```

dp_scale_encode	<i>Encode numeric columns into integer storage columns</i>
-----------------	--

Description

Creates new integer storage columns (default suffix `"_i"`) by multiplying numeric columns by their `'writer_scale'` in the spec.

Usage

```
dp_scale_encode(
  dt,
  spec,
  suffix = "_i",
  drop_original = FALSE,
  round_fn = c("round", "floor", "ceiling")
)
```

Arguments

dt	A 'data.frame' or 'data.table'.
spec	A 'dp_spec' describing scaling intent ('writer_scale').
suffix	Suffix for storage columns (default <code>"_i"</code>).
drop_original	Logical. If 'TRUE', drop the original float columns.
round_fn	Rounding method: <code>"round"</code> , <code>"floor"</code> , or <code>"ceiling"</code> .

Details

Intended use: - stable, compact integer storage while retaining predictable decimal precision - consistent round-trip with `[dp_scale_decode()]`

Value

A list with: - `'dt'`: a copied 'data.table' with storage columns added - `'scale_map'`: a 'data.table' describing what was scaled and where

Examples

```
## Not run:
library(data.table)
dt <- data.table(temp = c(25.12, 25.44))
sp <- dp_spec_default(dt, "metric")
sp <- dp_set_scale(sp, "temp", 100)
enc <- dp_scale_encode(dt, sp)
enc$dt

## End(Not run)
```

dp_set_scale	<i>Set scaling intent in a spec</i>
--------------	-------------------------------------

Description

Sets 'writer_scale' for selected columns in a 'dp_spec'.

Usage

```
dp_set_scale(spec, columns, writer_scale)
```

Arguments

spec	A 'dp_spec'.
columns	Character vector of column names to edit.
writer_scale	Numeric scalar scale factor (or 'NA' to clear).

Details

This does not change your data by itself; it records the intent used by [dp_scale_encode()] and by [dp_write(..., scale = TRUE)].

Value

Updated 'dp_spec'.

Examples

```
## Not run:
library(data.table)
dt <- data.table(temp = 25.12)
sp <- dp_spec_default(dt, "metric")
sp <- dp_set_scale(sp, "temp", 100) # keep 2 decimals as integer storage

## End(Not run)
```

dp_set_units	<i>Set units in a spec</i>
--------------	----------------------------

Description

Updates unit fields within a 'dp_spec' for one or more columns.

Usage

```
dp_set_units(
  spec,
  columns,
  metric_units = NULL,
  imperial_units = NULL,
  declared_units = NULL
)
```

Arguments

spec	A 'dp_spec'.
columns	Character vector of column names to edit.
metric_units	Optional character scalar unit to set for selected columns.
imperial_units	Optional character scalar unit to set for selected columns.
declared_units	Optional character scalar unit override to set for selected columns.

Details

Relationship to [dp_tag_units()]: - 'dp_set_units()' updates the **spec** (the contract / documentation) - 'dp_tag_units()' annotates an **in-memory R object** based on that spec

Value

Updated 'dp_spec'.

Examples

```
## Not run:
library(data.table)
dt <- data.table(wind = 2.1)
sp <- dp_spec_default(dt, "metric")
sp <- dp_set_units(sp, "wind", metric_units = "m/s", imperial_units = "mph")

## End(Not run)
```

dp_spec	<i>Dataplane spec objects</i>
---------	-------------------------------

Description

A 'dp_spec' describes columns: concepts, unit systems, and optional scaling intent ('writer_scale') used for integer storage encoding.

Alias for [dp_spec_default()].

Usage

```
dp_spec(x, declared_system = c("metric", "imperial"), include_unmatched = TRUE)
```

Arguments

x	A 'data.frame' or 'data.table' (only column names are used).
declared_system	Character scalar: "metric" or "imperial". This controls how 'declared_units' are inferred when not explicitly set.
include_unmatched	Logical. If 'TRUE' (default), keep all columns even if no concept match is found. If 'FALSE', only keep matched columns.

Details

The typical workflow: 1) Create a spec with [dp_spec_default()] (or [dp_spec()] as an alias) 2) Edit with [dp_set_units()] and/or [dp_set_scale()] 3) Use in I/O via [dp_write()] / [dp_read()] or for in-memory annotation via [dp_tag_units()] / [dp_check_units()].

Value

A ‘dp_spec’ object.

dp_spec_default	Create a default spec from a table
-----------------	------------------------------------

Description

Builds a ‘dp_spec’ describing the columns in ‘x’. Dataplane uses a small built-in concept catalog (synonyms) to populate known concepts/units/scales. Unmatched columns are left as ‘NA’ by design.

Usage

```
dp_spec_default(
  x,
  declared_system = c("metric", "imperial"),
  include_unmatched = TRUE
)
```

Arguments

x	A ‘data.frame’ or ‘data.table’ (only column names are used).
declared_system	Character scalar: “metric” or “imperial”. This controls how ‘declared_units’ are inferred when not explicitly set.
include_unmatched	Logical. If ‘TRUE’ (default), keep all columns even if no concept match is found. If ‘FALSE’, only keep matched columns.

Value

A ‘dp_spec’ object.

Examples

```
## Not run:
library(data.table)
dt <- data.table(temp = 25.1, rh = 55)
sp <- dp_spec_default(dt, declared_system = "metric")
sp
sp$fields[]

## End(Not run)
```

dp_tag_units

*Tag unit attributes onto columns***Description**

Attaches units as an attribute (default attribute name: "units") onto columns of an R table, based on a 'dp_spec'.

Usage

```
dp_tag_units(
  dt,
  spec,
  only_missing = TRUE,
  system = c("declared", "metric", "imperial"),
  unit_attr = "units"
)
```

Arguments

dt	A 'data.frame' or 'data.table'.
spec	A 'dp_spec' with a 'fields' table.
only_missing	Logical. If 'TRUE', only attach units when the attribute is currently missing.
system	Which unit system to tag: "declared" (default), "metric", or "imperial".
unit_attr	Attribute name to write (default "units").

Details

Intended use: - in-memory annotation to reduce accidental unit confusion during analysis - no file I/O

Value

A list with: - 'dt': a copied 'data.table' with attributes attached - 'did_tag': an audit 'data.table' describing what was tagged

Examples

```
## Not run:
library(data.table)
dt <- data.table(temp = 25.1)
sp <- dp_spec_default(dt, "metric")
res <- dp_tag_units(dt, sp)
attr(res$dt$temp, "units")

## End(Not run)
```

dp_write

*Write a Parquet file with Dataplane metadata***Description**

Writes ‘x’ to a Parquet file and embeds Dataplane metadata in Parquet schema key/value pairs. Optional behaviors: - tag missing unit attributes before writing (in-memory) - validate unit attributes against a spec (warning by default) - encode selected numeric columns into integer storage columns

Usage

```
dp_write(
  x,
  path,
  spec = c("metric", "imperial"),
  mode = c("document", "validate"),
  scale = FALSE,
  suffix = "_i",
  drop_original = FALSE,
  tag_missing_units = FALSE,
  prefix = "dp",
  compression = "zstd",
  unit_attr = "units",
  warn_max = 8L,
  ...
)
```

Arguments

x	A ‘data.frame’ or ‘data.table’.
path	Output ‘.parquet’ file path.
spec	Either a ‘dp_spec’ or one of “metric”, “imperial” (in which case a default spec is generated from column names).
mode	“document” or “validate”. If “validate”, runs [dp_check_units()] before writing (warns by default).
scale	Logical. If ‘TRUE’, apply [dp_scale_encode()] at write time.
suffix	Suffix for storage columns (default “_i”).
drop_original	Logical. If ‘TRUE’ and ‘scale=TRUE’, drop base columns after encoding.
tag_missing_units	Logical. If ‘TRUE’, apply [dp_tag_units()] before writing.
prefix	Metadata key prefix (default “dp”).
compression	Parquet compression (default “zstd”).
unit_attr	Unit attribute name to use (default “units”).
warn_max	Passed to [dp_check_units()].
...	Passed to [arrow::write_parquet()].

Value

Invisibly returns a list containing 'path', 'spec', 'audit', and 'kv'.

Examples

```
## Not run:
library(data.table)
dt <- data.table(temp = c(25.12, 25.44), rh = c(55, 52))
sp <- dp_spec_default(dt, "metric")
sp <- dp_set_scale(sp, "temp", 100)
out <- tempfile(fileext = ".parquet")
dp_write(dt, out, spec = sp, scale = TRUE)
res <- dp_read(out, attach_units = "declared")
res$dt

## End(Not run)
```

dp_write_dataset	<i>Write an Arrow Dataset and Dataplane sidecar metadata</i>
------------------	--

Description

Writes an Arrow Dataset (directory of Parquet files) and writes a Dataplane sidecar metadata Parquet file (default: 'dp_meta.parquet').

Usage

```
dp_write_dataset(
  x,
  path,
  spec = c("metric", "imperial"),
  scale_document = FALSE,
  suffix = "_i",
  prefix = "dp",
  sidecar_overwrite = TRUE,
  sidecar_compression = "zstd",
  write_format = "parquet",
  partitioning = NULL,
  existing_data_behavior = NULL,
  ...
)
```

Arguments

x	An Arrow Table, RecordBatchReader, Dataset, or object acceptable to [arrow::write_dataset()].
path	Dataset directory path.
spec	Either a 'dp_spec' or '"metric"/"imperial"' to create a default spec.
scale_document	Logical. If 'TRUE', document intended storage columns in metadata.
suffix	Storage suffix to document (default '"_i"').
prefix	Metadata prefix (default '"dp"').

sidecar_overwrite	Logical. Overwrite sidecar if present.
sidecar_compression	Compression used for the sidecar Parquet file.
write_format	Dataset format (currently only "parquet" supported here).
partitioning	Optional partitioning passed to [arrow::write_dataset()].
existing_data_behavior	Optional passed to [arrow::write_dataset()].
...	Additional args passed to [arrow::write_dataset()].

Details

Important: 'scale_document=TRUE' documents intended storage columns (based on 'writer_scale') but does not transform the dataset data. For actual scaling, use [dp_write()] per-file or pre-transform the table before dataset writing.

Value

Invisibly returns a list including dataset path and sidecar path.

dp_write_dataset_meta *Write Dataplane dataset metadata sidecar*

Description

Writes a tiny Parquet file in 'dataset_path' (default file name: 'dp_meta.parquet') containing Dataplane schema metadata in its Parquet schema key/value pairs.

Usage

```
dp_write_dataset_meta(
  dataset_path,
  kv,
  prefix = "dp",
  overwrite = TRUE,
  compression = "zstd",
  sidecar_name = NULL
)
```

Arguments

dataset_path	Dataset directory path.
kv	Named list of key/value metadata (character scalars recommended).
prefix	Prefix used for default sidecar name (default "dp").
overwrite	Logical. If 'FALSE' and sidecar exists, stop.
compression	Parquet compression for the sidecar (default "zstd").
sidecar_name	Optional override for sidecar file name.

Details

This provides a stable, single place to read dataset-level metadata without scanning all data files.

Value

The sidecar path (invisibly).

Examples

```
## Not run:  
kv <- list("dp:writer"="example")  
dp_write_dataset_meta("dataset_dir", kv)  
  
## End(Not run)
```

drop_null	<i>Remove NULL elements from a list</i>
-----------	---

Description

Remove NULL elements from a list

Usage

```
drop_null(alist)
```

Arguments

alist A list.

Value

A list with all 'NULL' entries removed.

Examples

```
drop_null(list(a = 1, b = NULL, c = 3))
```

env_close

*Gracefully Stop the Watchdog and Cluster***Description**

The 'env_close()' function stops the watchdog by creating the stop file, then uses 'stop_cluster()' to kill the associated Rscript.exe processes.

Usage

```
env_close(env_info, remove_watchdog_stopfile = TRUE)
```

Arguments

env_info A list as returned by 'env_start()', containing 'cluster', 'stop_file', and 'worker_pids'.

remove_watchdog_stopfile Boolean. If true, removes stopfile corresponding to this watchdog/environment. Default is TRUE.

Details

This function is useful for a clean shutdown after your computations are finished. It signals the watchdog script to stop by creating the specified stop file, and then forcefully kills the worker processes to ensure a clean slate.

After calling 'env_close()', if you also wish to stop the parallel cluster, call: `parallel::stopCluster(env_info$cluster)`

Value

NULL (invisibly).

Examples

```
## Not run:
env_info <- env_start(num_cores = 2)

# ... run computations ...

env_close(env_info)
# Now the watchdog should stop and the processes are terminated.

parallel::stopCluster(env_info$cluster)

## End(Not run)
```

env_start	<i>Start a Parallel Cluster with a Cross-Platform Watchdog (Temp Stop File)</i>
-----------	---

Description

‘env_start()’ sets up a parallel cluster and starts a watchdog script that monitors memory usage on Windows or Linux. It writes out default watchdog scripts if needed.

If ‘stop_file’ is not provided, a unique temporary filename (with random characters) is generated.

Usage

```
env_start(
  num_cores = 2,
  threshold = 0.99,
  consecutive_seconds = 15,
  stop_file = tempfile("stop_watchdog_"),
  windows_path = "C:\\Users\\Jordan\\Desktop",
  linux_path = "~/scripts"
)
```

Arguments

num_cores	Integer. Number of cores to use for the parallel cluster. Defaults to 2.
threshold	Numeric. The memory usage threshold (0 to 1) at which the watchdog triggers. Default 0.99.
consecutive_seconds	Integer. How many consecutive seconds memory must be above threshold before killing. Default 15.
stop_file	Character. The full path and filename of the stop file to be used by the watchdog. If not specified, a random unique temp file name is created.
windows_path	Character. The directory for ‘watchdog2.ps1’ on Windows. Defaults to “C:\Users\Jordan\Desktop”.
linux_path	Character. The directory for ‘watchdog2.sh’ on Linux. Defaults to “~/scripts”.

Details

- On Windows, uses a PowerShell script (‘watchdog2.ps1’). - On Linux, uses a Bash script (‘watchdog2.sh’). - If the watchdog script doesn’t exist, it is written out. - If ‘stop_file’ is ‘NULL’, a unique temp file is created automatically.

The watchdog monitors memory usage. If it stays above ‘threshold’ for ‘consecutive_seconds’, it kills the provided worker PIDs. To stop the watchdog gracefully, create the stop file specified by ‘stop_file’.

Value

A list: ‘list(cluster = cl, stop_file = stop_file, worker_pids = pids)’.

Examples

```
## Not run:
# Start environment with a random unique stop file name:
env_info <- env_start()

# ... do some computations ...

# Gracefully stop watchdog:
# ?jj::env_close
env_close(env_info)
file.create(env_info$stop_file)
stop_cluster(env_info$worker_pids)
# file.create(env_info$stop_file)
# Kill processes if desired:
stop_cluster(env_info$worker_pids)
# Then stop cluster (probably DO NOT need to also do this after calling stop_cluster():
parallel::stopCluster(env_info$cluster)

## End(Not run)
```

filter_cols

Filter column names by storage type

Description

Returns the column names whose **primary class** matches a target set, or (if `'exclude = TRUE'`) those that do not.

Usage

```
filter_cols(dat, colType = "numeric", exlcluedatetime = TRUE, exclude = FALSE)
```

Arguments

dat	A data.frame / data.table.
colType	Character. Either <code>"numeric"</code> (default) or a character vector of class names (e.g., <code>'c("character")'</code>).
exlcluedatetime	Logical. If <code>'TRUE'</code> , columns with primary class in <code>'c("Date","POSIXct","POSIXt")'</code> are excluded from the match set.
exclude	Logical. If <code>'TRUE'</code> , returns columns <i>*not*</i> matching <code>'colType'</code> .

Value

Character vector of column names.

Examples

```
d <- data.frame(x = 1:3, y = c("a","b","c"), t = as.POSIXct("2026-01-01") + 1:3)
filter_cols(d, "numeric")
filter_cols(d, "character")
filter_cols(d, "numeric", exclude = TRUE)
```

`flatten_list_columns` *Flatten list-columns into character columns*

Description

For any list-column, converts each element to a single string:

- 'NULL' -> 'NA'
- length-1 -> that value
- length>1 -> comma-separated string

Usage

```
flatten_list_columns(dt)
```

Arguments

`dt` A data.frame / tibble / data.table-like object.

Value

An object of the same class as 'dt', with list-columns flattened.

Examples

```
x <- data.frame(a = 1:3, b = I(list("x", c("y", "z"), NULL)))
flatten_list_columns(x)
```

`gp_geoparquet_attach_geo_to_table`
Attach GeoParquet metadata to an Arrow Table

Description

Adds a 'geo' JSON metadata entry to an Arrow Table's schema metadata.

Usage

```
gp_geoparquet_attach_geo_to_table(tbl, geo_meta, geo_key = "geo")
```

Arguments

`tbl` An Arrow Table.
`geo_meta` GeoParquet metadata list.
`geo_key` Metadata key to store geo JSON (default "'geo'").

Value

An Arrow Table with updated schema metadata.

gp_geoparquet_build_geo_meta

Build a GeoParquet 'geo' metadata object

Description

Constructs a GeoParquet-compliant metadata list for one or more geometry columns. This metadata can be serialized to JSON and stored in Parquet schema metadata under the key "geo" (the GeoParquet convention).

Usage

```
gp_geoparquet_build_geo_meta(
  geometry_columns,
  primary_column = geometry_columns[1],
  encoding = "WKB",
  geometry_types = NULL,
  crs = NULL,
  edges = c("planar", "spherical"),
  orientation = c(NULL, "counterclockwise"),
  bbox = NULL,
  covering_bbox = NULL,
  version = "1.1.0",
  extra = list()
)
```

Arguments

geometry_columns	Character vector of geometry column names.
primary_column	Primary geometry column name (defaults to first geometry column).
encoding	Geometry encoding (default "WKB").
geometry_types	Optional geometry type(s). May be: - a character vector applied to all geometry columns, or - a named list keyed by geometry column.
crs	CRS as PROJJSON (list) or a string convertible via <code>sf::st_crs()</code> .
edges	"planar" or "spherical".
orientation	Optional orientation string (currently supports "counterclockwise").
bbox	Optional numeric bounding box.
covering_bbox	Optional covering bbox spec (see GeoParquet).
version	GeoParquet metadata version (default "1.1.0").
extra	Optional additional fields to include in the metadata list.

Value

A nested list suitable for JSON serialization via `[gp_geoparquet_geo_json()]`.

gp_geoparquet_geo_json

Serialize GeoParquet metadata to JSON

Description

Serialize GeoParquet metadata to JSON

Usage

```
gp_geoparquet_geo_json(geo_meta)
```

Arguments

geo_meta A metadata list created by [gp_geoparquet_build_geo_meta()].

Value

A JSON string.

gp_geoparquet_infer_from_sf

Infer GeoParquet metadata from an sf object

Description

Convenience helper that infers geometry types and CRS PROJJSON from an 'sf' object and returns a GeoParquet metadata list.

Usage

```
gp_geoparquet_infer_from_sf(
  sf_obj,
  geometry_column = NULL,
  encoding = "WKB",
  version = "1.1.0",
  edges = "planar",
  orientation = NULL,
  covering_bbox = NULL,
  extra = list()
)
```

Arguments

sf_obj An 'sf' object.

geometry_column Geometry column name (defaults to 'attr(sf_obj, "sf_column")').

encoding Geometry encoding (default "WKB").

version GeoParquet metadata version (default "1.1.0").

edges	"planar" or "spherical" (default "planar").
orientation	Optional orientation string.
covering_bbox	Optional covering bbox spec.
extra	Optional additional fields to include.

Value

A GeoParquet metadata list suitable for [gp_geoparquet_geo_json()].

Examples

```
## Not run:
library(sf)
x <- st_as_sf(data.frame(id=1:2, x=c(0,1), y=c(0,1)), coords=c("x","y"), crs=4326)
meta <- gp_geoparquet_infer_from_sf(x)

## End(Not run)
```

gp_geoparquet_is_geoparquet

Test whether a Parquet file is GeoParquet

Description

Test whether a Parquet file is GeoParquet

Usage

```
gp_geoparquet_is_geoparquet(path, geo_key = "geo")
```

Arguments

path	Parquet file path.
geo_key	Metadata key holding geo JSON (default "geo").

Value

Logical.

`gp_geoparquet_read_geo`*Read GeoParquet 'geo' metadata from a Parquet file*

Description

Read GeoParquet 'geo' metadata from a Parquet file

Usage

```
gp_geoparquet_read_geo(path, geo_key = "geo")
```

Arguments

<code>path</code>	Parquet file path.
<code>geo_key</code>	Metadata key holding geo JSON (default "geo").

Value

Parsed GeoParquet metadata list, or 'NULL' if not present.

`gp_geoparquet_write_parquet`*Write GeoParquet with GeoParquet metadata*

Description

Writes a Parquet file with GeoParquet 'geo' metadata embedded in schema metadata.

Usage

```
gp_geoparquet_write_parquet(x, path, geo_meta, compression = "zstd", ...)
```

Arguments

<code>x</code>	Data frame-like object to write (often an 'sf' object after dropping geometry list-columns).
<code>path</code>	Output Parquet file path.
<code>geo_meta</code>	GeoParquet metadata list.
<code>compression</code>	Parquet compression (default "zstd").
<code>...</code>	Passed to <code>[arrow::write_parquet()]</code> .

Value

The output path (invisibly).

`mavtime`*Calculate Rolling Averages for Time-Series Data*

Description

The ‘mavtime’ function calculates rolling averages over a specified time window for time-series data. This implementation leverages an optimized Rcpp function for enhanced performance on large datasets.

Usage

```
mavtime(  
  timevals,  
  datavals,  
  timewindow,  
  timeunits = "min",  
  resample = NULL,  
  resampleinterval = "1 min"  
)
```

Arguments

<code>timevals</code>	A ‘POSIXct’ vector of timestamps for the data points.
<code>datavals</code>	A numeric vector of data values corresponding to the timestamps.
<code>timewindow</code>	An integer specifying the size of the rolling window in the units defined by ‘timeunits’.
<code>timeunits</code>	A character string specifying the time units for the rolling window. Supported values are: “min” (default), “hour”, or “day”.
<code>resample</code>	A logical indicating whether to resample the data to regular intervals (default: ‘TRUE’).
<code>resampleinterval</code>	A character string specifying the interval for resampling, e.g., “1 min”.

Details

This function resamples the data to regular intervals if ‘resample = TRUE’, ensuring consistent time-series alignment before calculating rolling averages. The rolling averages are computed using a highly optimized Rcpp implementation for performance.

Value

A numeric vector of rolling averages aligned with the input ‘timevals’. Missing values (‘NA’) are returned if insufficient data exists within the window.

See Also

‘rollapply’ from the zoo package for a pure R implementation.

Examples

```
library(data.table)

# Generate example time-series data
timevals <- seq(from = as.POSIXct("2024-12-01 00:00:00"), by = "1 min", length.out = 100)
datavals <- runif(100, min = 0, max = 100)

# Calculate rolling averages with a 15-minute window
result <- mavtime(timevals, datavals, timewindow = 15, timeunits = "min")
```

merge_dt_list	Merge a list of data.frames/data.tables by keys
---------------	---

Description

Iteratively merges a list of tabular objects using ‘merge(..., all = TRUE)’.

Usage

```
merge_dt_list(alist, by)
```

Arguments

- alist List of data.frames / data.tables.
- by Character vector of key columns passed to ‘merge()’.

Value

A merged data.frame/data.table (depends on inputs).

Examples

```
a <- data.frame(id = 1:3, x = 11:13)
b <- data.frame(id = 2:4, y = 21:23)
merge_dt_list(list(a, b), by = "id")
```

null_coalesce	Null-coalescing operator
---------------	--------------------------

Description

Null-coalescing operator

Usage

```
a %||% b
```

print.dp_spec	<i>Print a Dataplane spec</i>
---------------	-------------------------------

Description

Compact printer for ‘dp_spec’ objects.

Usage

```
## S3 method for class 'dp_spec'
print(x, ...)
```

Arguments

x	A ‘dp_spec’.
...	Unused.

Value

The input ‘x’ (invisibly).

rep_var	<i>Repeat a scalar to length n (or recycle shorter vectors)</i>
---------	---

Description

Utility for aligning scalar parameters with vectorized data.

Usage

```
rep_var(var, n)
```

Arguments

var	A vector or ‘NULL’.
n	Integer target length.

Value

‘NULL’ if ‘var’ is ‘NULL’; otherwise a vector of length ‘n’ (or the original vector if already length ‘n’ or longer). If ‘length(var) < n’, the value is recycled with a warning.

Examples

```
rep_var(NULL, 5)
rep_var(3, 5)
rep_var(1:2, 5)
```

require_ram	<i>Block Execution Until Available RAM Exceeds Threshold</i>
-------------	--

Description

require_ram pauses execution in a loop until the system reports at least the specified amount of free RAM. Useful for long-running data-intensive tasks that require guaranteed memory availability before proceeding.

Usage

```
require_ram(threshold = 4000, unit = "MB", wait_time = 10, max_attempts = 10)
```

Arguments

threshold	Numeric. Minimum amount of free memory required (default 4000).
unit	Character. Unit for memory threshold (e.g., "MB" or "GB").
wait_time	Numeric. Seconds to wait between consecutive checks (default 10).
max_attempts	Integer. Maximum number of check attempts before giving up (default 10).

Details

This function relies on check_ram() to query system memory. It will repeatedly sleep for wait_time seconds and retry until the available RAM is sufficient or the maximum number of attempts is reached.

Value

Invisibly returns TRUE once the available RAM meets or exceeds the threshold.

Examples

```
## Not run:
# Wait for at least 4 GB free memory before starting heavy computation
require_ram(threshold = 4000, unit = "MB", wait_time = 10, max_attempts = 20)

## End(Not run)
```

round_cols	<i>Round selected numeric columns</i>
------------	---------------------------------------

Description

Rounds specified columns in a data.frame or data.table.

Usage

```
round_cols(x, cols, digits = 2, in_place = FALSE)
```

Arguments

<code>x</code>	A data.frame or data.table.
<code>cols</code>	Character vector of column names to round.
<code>digits</code>	Integer. Number of decimal places passed to <code>'round()'</code> .
<code>in_place</code>	Logical. If <code>'TRUE'</code> and <code>'x'</code> is a data.table, modifies <code>'x'</code> by reference. Otherwise returns a modified copy.

Value

The modified object (invisibly returns `'x'` when `'in_place = TRUE'`).

Examples

```
df <- data.frame(a = 1:5/2.1, b = 6:10/1.59)
round_cols(df, cols = c("a", "b"), digits = 2)

## Not run:
library(data.table)
dt <- data.table(a = 1:5/2.1, b = 6:10/1.59)
round_cols(dt, c("a", "b"), 2, in_place = TRUE)

## End(Not run)
```

<code>s3_create_dir</code>	<i>Create an "S3 directory" placeholder</i>
----------------------------	---

Description

S3 does not have real directories; prefixes ending in `'/'` are treated like folders by many tools. This function checks whether any objects already exist under a prefix and, if not, creates a zero-byte placeholder object at that prefix.

Usage

```
s3_create_dir(bucket_name, directory_name)
```

Arguments

<code>bucket_name</code>	Character scalar. S3 bucket name.
<code>directory_name</code>	Character scalar. Prefix to create. A trailing <code>'/'</code> is added if missing.

Value

Invisibly returns `'TRUE'` if a placeholder object was created, `'FALSE'` if the prefix already had objects.

Examples

```
## Not run:
s3_create_dir("my-bucket", "some/prefix")
s3_create_dir("my-bucket", "some/prefix/") # equivalent

## End(Not run)
```

s3_download_file	<i>Download an R object from S3 (stored as RDS)</i>
------------------	---

Description

Downloads an object from S3 to a temporary file and reads it with ‘readRDS()’.

Usage

```
s3_download_file(key, bucket, fileext = ".rds")
```

Arguments

key	Character scalar. S3 object key (path) to download.
bucket	Character scalar. S3 bucket name.
fileext	Character scalar. Temporary file extension. Defaults to ‘.rds’.

Value

The R object stored in the RDS.

Examples

```
## Not run:
x <- s3_download_file("path/to/object.rds", bucket = "my-bucket")

## End(Not run)
```

s3_list_objects	<i>List S3 objects with retries and optional ordering</i>
-----------------	---

Description

Wrapper around ‘aws.s3::get_bucket()’ that:

- supports basic pagination via ‘marker’
- retries transient failures with backoff
- returns a data frame with ‘Key’, ‘Size’, ‘LastModified’, ‘StorageClass’
- can order by ‘LastModified’ and/or return only the tail rows

Lists objects from an S3 bucket/prefix using ‘aws.s3’, avoiding the ‘Owner\$ID’ assumption that can trigger ‘subscript out of bounds’ when ‘Owner’ is omitted. Adds retries and (optionally) sorts by ‘LastModified’ and returns only the newest rows.

Usage

```
s3_list_objects(
  bucket,
  prefix = NULL,
  delimiter = NULL,
  max = NULL,
  marker = NULL,
  ...,
  max_tries = 3,
  wait = 1,
  backoff = 1.6,
  verbose = TRUE,
  order_by_last_modified = FALSE,
  return_tail = FALSE,
  tail_n = 6
)
```

```
s3_list_objects(
  bucket,
  prefix = NULL,
  delimiter = NULL,
  max = NULL,
  marker = NULL,
  ...,
  max_tries = 3,
  wait = 1,
  backoff = 1.6,
  verbose = TRUE,
  order_by_last_modified = FALSE,
  return_tail = FALSE,
  tail_n = 6
)
```

Arguments

bucket	Character. S3 bucket name.
prefix	Character, optional. Limit results to keys beginning with this prefix.
delimiter	Character, optional. E.g., "/" to group by common prefixes.
max	Integer, optional. Maximum number of keys to return (per request).
marker	Character, optional. Key to start with when listing.
...	Passed through to [aws.s3::get_bucket()] (e.g., credentials, region).
max_tries	Integer. Maximum attempts before failing. Default '3'.
wait	Numeric (seconds). Initial wait between attempts. Default '1'.
backoff	Numeric multiplier for exponential backoff. Default '1.6'.
verbose	Logical. If 'TRUE', prints retry messages. Default 'TRUE'.
order_by_last_modified	Logical. If 'TRUE', sort ascending by 'LastModified'. Default 'FALSE'.
return_tail	Logical. If 'TRUE', return only the last 'tail_n' rows *after* optional sorting. Default 'FALSE'.
tail_n	Integer. Number of rows to return when 'return_tail = TRUE'. Default '6'.

Details

Calls `[aws.s3::get_bucket()]` with `'parse_response = TRUE'` and converts the parsed list into a data frame while ignoring any `'Owner'` fields. Handles "folder" entries by falling back to `'Name'` or `'Prefix'` when `'Key'` is absent. Timestamps are parsed to UTC when present.

Note: this issues a single `ListObjects` request (no pagination). For >1,000 keys, loop with `'marker'/continuation` tokens.

Value

A data.frame with columns: `'Key'`, `'Size'`, `'LastModified'`, `'StorageClass'`.

A `'data.frame'` with columns: - `'Key'` (character) - `'Size'` (numeric; `'NA'` for common prefixes) - `'LastModified'` (POSIXct, UTC; `'NA'` if absent) - `'StorageClass'` (character; `'NA'` if absent)

See Also

`[aws.s3::get_bucket()]`, `[aws.s3::save_object()]`

Examples

```
## Not run:
df <- s3_list_objects(bucket = "my-bucket", prefix = "data/2026-01-01/")
newest <- s3_list_objects("my-bucket", prefix = "data/", order_by_last_modified = TRUE, return_tail = TRUE, ta

## End(Not run)

## Not run:
# Basic
df <- safe_get_bucket_df("my-bucket", prefix = "hrrr/20250809/")

# Newest few objects (sorted by LastModified, tail 6)
newest <- safe_get_bucket_df("my-bucket", prefix = "hrrr/20250809/",
                             order_by_last_modified = TRUE, return_tail = TRUE, tail_n = 6)

## End(Not run)
```

s3_upload_file

Save an R object to S3 (as RDS)

Description

Serializes an R object via `'saveRDS()'` into a temporary file, uploads it to S3, then removes the temporary file.

Usage

```
s3_upload_file(object, bucket, s3_path, temp_dir = tempdir())
```

Arguments

object	R object to serialize.
bucket	Character scalar. Bucket name.
s3_path	Character scalar. Destination S3 key (path).
temp_dir	Character scalar. Directory to place the temporary file.

Value

The result returned by 'aws.s3::put_object()'.

Examples

```
## Not run:
res <- s3_upload_file(list(a = 1), bucket = "my-bucket", s3_path = "tmp/test.rds")

## End(Not run)
```

sanitize_value	<i>Sanitize a scalar value for SQL-like string construction</i>
----------------	---

Description

Converts a single value into a form often used when building SQL statements:

- 'NA' -> "NULL" (character)
- logical -> '0/1' (integer)
- character/Date/POSIXct -> single-quoted, with internal '' escaped
- numeric -> returned as-is

Usage

```
sanitize_value(x)
```

Arguments

x A length-1 value.

Value

A length-1 value suitable for pasting into query text.

Examples

```
sanitize_value(NA)
sanitize_value(TRUE)
sanitize_value("O'Reilly")
sanitize_value(as.Date("2026-01-12"))
sanitize_value(3.14)
```

stop_cluster

Forcefully Stop Worker Processes

Description

'stop_cluster()' attempts to kill the processes by PID. On Windows, uses 'taskkill /F /PID'. On Linux, uses 'kill -9'.

Usage

```
stop_cluster(workerpids = NULL)
```

Arguments

workerpids Integer vector of worker PIDs. If 'NULL', kills all 'Rscript.exe' (Windows) or 'R' (Linux) processes. Use with caution.

Value

NULL invisibly.

Examples

```
## Not run:
stop_cluster(env_info$worker_pids)

## End(Not run)
```

syn_create_file_entity

Create a FileEntity under a parent container

Description

Typically used after creating a file handle via multipart upload.

Usage

```
syn_create_file_entity(
  parent_id,
  name,
  dataFileHandleId,
  description = NULL,
  token = NULL,
  verbose = FALSE,
  dry_run = FALSE
)
```

Arguments

parent_id	Parent Synapse ID (Project or Folder).
name	FileEntity name.
dataFileHandleId	File handle ID returned by multipart upload completion.
description	Optional description string.
token	Optional token (PAT).
verbose	Logical.
dry_run	Logical.

Value

Parsed JSON list for created entity; contains 'id'.

Examples

```
## Not run:
fe <- syn_create_file_entity("synFolderId", "data.csv", dataFileHandleId = "123456")
fe$id

## End(Not run)
```

syn_create_folder	<i>Create a Folder under a parent container</i>
-------------------	---

Description

Create a Folder under a parent container

Usage

```
syn_create_folder(
  parent_id,
  name,
  description = NULL,
  token = NULL,
  verbose = FALSE,
  dry_run = FALSE
)
```

Arguments

parent_id	Parent Synapse ID (Project or Folder).
name	Folder name (single path segment).
description	Optional description string.
token	Optional token (PAT).
verbose	Logical.
dry_run	Logical.

Value

Parsed JSON list for created entity; contains 'id'.

Examples

```
## Not run:
f <- syn_create_folder("synProjectId", "my_folder", description = "Created via REST")
f$id

## End(Not run)
```

syn_download_file	<i>Download a Synapse FileEntity to disk</i>
-------------------	--

Description

Fetches the FileEntity to obtain 'dataFileHandleId', resolves a presigned URL via the file handle URL endpoint, and streams the file content to 'dest_path'.

Usage

```
syn_download_file(
  entity_id,
  dest_path,
  overwrite = FALSE,
  token = NULL,
  verbose = FALSE,
  dry_run = FALSE
)
```

Arguments

entity_id	FileEntity synId.
dest_path	Destination path (will create parent directories).
overwrite	Logical; overwrite existing file if 'TRUE'.
token	Optional token (PAT).
verbose	Logical.
dry_run	Logical.

Value

'dest_path' (invisibly in dry-run, otherwise returned as a character scalar).

Examples

```
## Not run:
dest <- tempfile(fileext = ".csv")
syn_download_file("synFileEntityId", dest, overwrite = TRUE, verbose = TRUE)
read.csv(dest)

## End(Not run)
```

 syn_download_file_path

Download a file using a remote path (relative to a base container)

Description

Download a file using a remote path (relative to a base container)

Usage

```
syn_download_file_path(
  base_id,
  remote_file_path,
  dest_path,
  overwrite = FALSE,
  token = NULL,
  verbose = FALSE,
  dry_run = FALSE
)
```

Arguments

base_id	Base container synId.
remote_file_path	Remote file path like "a/b/c/file.csv".
dest_path	Local destination path.
overwrite	Logical; overwrite existing local file.
token	Optional token (PAT).
verbose	Logical.
dry_run	Logical.

Value

'dest_path'.

Examples

```
## Not run:
syn_download_file_path(
  base_id = "synProjectId",
  remote_file_path = "zz_smoketests/synapse_rw/test2/New Text Document.txt",
  dest_path = "C:/tmp/downloads/test.txt",
  overwrite = TRUE,
  verbose = TRUE
)

## End(Not run)
```

syn_ensure_folder_path

Ensure a folder path exists under a base container

Description

Ensures each segment exists as a Folder under 'parent_id'. Missing folders are created. Existing segments are validated as Folders (not files) by calling [syn_get_entity()] on each resolved synId.

Usage

```
syn_ensure_folder_path(
  parent_id,
  path,
  token = NULL,
  verbose = FALSE,
  dry_run = FALSE
)
```

Arguments

parent_id	Base container synId (Project or Folder).
path	Remote folder path like "a/b/c" (slashes only; leading/trailing ignored).
token	Optional token (PAT).
verbose	Logical.
dry_run	Logical.

Value

synId of the deepest folder in the path.

Examples

```
## Not run:
folder_id <- syn_ensure_folder_path("synProjectId", "analysis/run_01")
folder_id

## End(Not run)
```

syn_get_annotations

Get annotations2 for an entity

Description

Get annotations2 for an entity

Usage

```
syn_get_annotations(entity_id, token = NULL, verbose = FALSE, dry_run = FALSE)
```

Arguments

entity_id	synId.
token	Optional token (PAT).
verbose	Logical.
dry_run	Logical.

Value

Parsed JSON list containing 'etag' and 'annotations'.

Examples

```
## Not run:
ann <- syn_get_annotations("synEntityId")
names(ann$annotations)

## End(Not run)
```

syn_get_child_by_name *Get a child entity by name under a parent container*

Description

Wraps 'POST /repo/v1/entity/child'.

Usage

```
syn_get_child_by_name(
  parent_id,
  name,
  include_types = NULL,
  token = NULL,
  verbose = FALSE,
  dry_run = FALSE
)
```

Arguments

parent_id	Parent synId.
name	Child name.
include_types	Optional vector of type filters (e.g., "folder", "file").
token	Optional token (PAT).
verbose	Logical.
dry_run	Logical.

Value

Parsed JSON list including 'id' and (depending on response) type metadata.

Examples

```
## Not run:
hit <- syn_get_child_by_name("synParent", "my_folder", include_types = "folder")
hit$id

## End(Not run)
```

syn_get_entity	<i>Get a Synapse entity</i>
----------------	-----------------------------

Description

Get a Synapse entity

Usage

```
syn_get_entity(entity_id, token = NULL, verbose = FALSE, dry_run = FALSE)
```

Arguments

entity_id	Synapse ID (e.g., "syn123").
token	Optional token (PAT).
verbose	Logical; emit request progress messages.
dry_run	Logical; do not execute.

Value

Parsed JSON list describing the entity.

Examples

```
## Not run:
e <- syn_get_entity("syn123")
e$concreteType

## End(Not run)
```

syn_lookup_child_id	<i>Lookup a child synId by parent + name</i>
---------------------	--

Description

Convenience wrapper returning just the child 'id' or 'NULL' if not found.

Usage

```
syn_lookup_child_id(
  parent_id,
  entity_name,
  token = NULL,
  verbose = FALSE,
  dry_run = FALSE
)
```

Arguments

parent_id	Parent synId.
entity_name	Child name.
token	Optional token (PAT).
verbose	Logical.
dry_run	Logical.

Value

Child synId character scalar, or 'NULL'.

Examples

```
## Not run:
cid <- syn_lookup_child_id("synParent", "child_name")

## End(Not run)
```

syn_request

Perform a Synapse REST request

Description

Low-level HTTP wrapper around Synapse endpoints using **httr2**.

Usage

```
syn_request(
  method,
  path,
  query = NULL,
  body = NULL,
  body_json = TRUE,
  headers = list(),
  token = NULL,
  retry = TRUE,
  verbose = FALSE,
  dry_run = FALSE
)
```

Arguments

method	HTTP method (e.g., "GET", "POST", "PUT").
path	Path starting with '/' (e.g., "/repo/v1/entity/syn123").
query	Optional named list of query parameters.
body	Optional request body.
body_json	If 'TRUE', encode body as JSON and set content-type accordingly.
headers	Optional named list of extra headers.
token	Optional Synapse token (PAT). If 'NULL', env vars are used.
retry	Logical; retry transient failures when 'TRUE'.
verbose	Logical; emit request/response progress messages.
dry_run	Logical; do not execute the request, return a description instead.

Details

- Adds Authorization header using a PAT (see [syn_resolve_token()]). - Retries transient failures (network, HTTP 429, HTTP 5xx) with exponential backoff. - Parses JSON responses when content-type indicates JSON; otherwise returns text.

Value

For JSON: a list (parsed JSON). For non-JSON: a character string body.

Examples

```
## Not run:
tok <- syn_resolve_token()
ent <- syn_request("GET", "/repo/v1/entity/syn123", token = tok)
str(ent)

## End(Not run)
```

syn_resolve_entity_by_path

Resolve an entity synId from a remote path under a base container

Description

- If 'remote_path' ends with '/', the last segment must resolve to a Folder. - Otherwise the last segment may resolve to either a Folder or FileEntity.

Usage

```
syn_resolve_entity_by_path(
  base_id,
  remote_path,
  token = NULL,
  verbose = FALSE,
  dry_run = FALSE
)
```

Arguments

base_id	Base container synId (Project or Folder).
remote_path	Remote path like <code>"a/b/file.csv"</code> or <code>"a/b/folder/"</code> .
token	Optional token (PAT).
verbose	Logical.
dry_run	Logical.

Details

Folder-ness checks are enforced by calling `[syn_get_entity()]` on intermediate segments.

Value

synId of the resolved entity.

Examples

```
## Not run:
eid <- syn_resolve_entity_by_path("synProjectId", "a/b/c/myfile.csv")
eid

## End(Not run)
```

syn_resolve_token	<i>Resolve a Synapse Personal Access Token (PAT)</i>
-------------------	--

Description

Returns a token from (in order): 1) `'token'` argument (if a long-ish scalar string) 2) `'Sys.getenv("SYNAPSE_PAT")'` (preferred) 3) `'Sys.getenv("SYNAPSE_AUTH_TOKEN")'` (fallback)

Usage

```
syn_resolve_token(token = NULL)
```

Arguments

token	Optional token string.
-------	------------------------

Value

A character scalar token.

Examples

```
## Not run:
Sys.setenv(SYNAPSE_PAT = "YOUR_PAT")
tok <- syn_resolve_token()

## End(Not run)
```

syn_set_annotations	<i>Set (merge) annotations2 on an entity</i>
---------------------	--

Description

Fetches current annotations2, merges keys provided in ‘annotations’ (overwriting those keys), and PUTs the updated annotations back using the current ‘etag’.

Usage

```
syn_set_annotations(  
  entity_id,  
  annotations,  
  token = NULL,  
  verbose = FALSE,  
  dry_run = FALSE  
)
```

Arguments

entity_id	synId.
annotations	Named list of annotations. Values may be scalar or vectors.
token	Optional token (PAT).
verbose	Logical.
dry_run	Logical.

Value

Parsed JSON list from the PUT request.

Examples

```
## Not run:  
syn_set_annotations("synEntityId", list(stage = "raw", tags = c("a","b")))  
  
## End(Not run)
```

syn_try_get_child_by_name	<i>Try to get a child entity by name (returns NULL on 404)</i>
---------------------------	--

Description

Try to get a child entity by name (returns NULL on 404)

Usage

```
syn_try_get_child_by_name(
  parent_id,
  name,
  include_types = NULL,
  token = NULL,
  verbose = FALSE,
  dry_run = FALSE
)
```

Arguments

parent_id	Parent synId.
name	Child name.
include_types	Optional vector of type filters (e.g., "folder", "file").
token	Optional token (PAT).
verbose	Logical.
dry_run	Logical.

Value

Parsed JSON list, or 'NULL' if not found.

Examples

```
## Not run:
hit <- syn_try_get_child_by_name("synParent", "maybe_exists", include_types = c("folder", "file"))
if (is.null(hit)) message("Not found")

## End(Not run)
```

syn_update_file_entity

Update a FileEntity to point to a new file handle

Description

Update a FileEntity to point to a new file handle

Usage

```
syn_update_file_entity(
  entity_id,
  new_dataFileHandleId,
  token = NULL,
  verbose = FALSE,
  dry_run = FALSE
)
```


Arguments

entity_id	FileEntity synId.
new_dataFileHandleId	New file handle ID.
token	Optional token (PAT).
verbose	Logical.
dry_run	Logical.

Value

Parsed JSON list for updated entity.

Examples

```
## Not run:
syn_update_file_entity("synFileEntityId", new_dataFileHandleId = "999999")

## End(Not run)
```

syn_upload_file	<i>Upload a local file to Synapse (multipart) and create/update a FileEntity</i>
-----------------	--

Description

Implements Synapse multipart upload: 1) Determine upload destination storage location for 'parent_id' 2) Initiate multipart upload 3) Upload parts to presigned S3 URLs and 'add' each part 4) Complete multipart upload to obtain a file handle ID 5) Create a new FileEntity under 'parent_id' or update an existing file with the same name 6) Optionally set annotations (annotations2)

Usage

```
syn_upload_file(
  local_path,
  parent_id,
  name = NULL,
  contentType = NULL,
  md5 = TRUE,
  overwrite = FALSE,
  description = NULL,
  annotations = NULL,
  token = NULL,
  verbose = FALSE,
  dry_run = FALSE
)
```

Arguments

local_path	Path to an existing local file.
parent_id	Parent container synId (Project or Folder).
name	Optional name for the FileEntity (defaults to 'basename(local_path)').
contentType	Optional MIME type (defaults to [tools::file_ext()] guess).
md5	Logical; compute MD5 for local file and parts (requires digest).
overwrite	Logical; if a file with the same name exists, update it.
description	Optional description for the FileEntity.
annotations	Optional named list of annotations (applied via annotations2).
token	Optional Synapse token (PAT).
verbose	Logical; emit progress.
dry_run	Logical; do not execute network requests.

Value

An object of class "syn_upload_result" with fields: - 'file_entity_id' - 'file_handle_id' - 'upload_id' - 'md5_hex' (file MD5 if computed) - 'size_bytes' - 'etags' (list of part ETags when available)

Examples

```
## Not run:
tmp <- tempfile(fileext = ".csv")
write.csv(data.frame(x=1:3, y=letters[1:3]), tmp, row.names = FALSE)

res <- syn_upload_file(
  local_path = tmp,
  parent_id = "synProjectOrFolder",
  overwrite = TRUE,
  annotations = list(stage = "raw", kind = "csv"),
  verbose = TRUE
)
res$file_entity_id

## End(Not run)
```

syn_upload_file_path *Upload a file to a remote folder path (relative to a base container)*

Description

Convenience wrapper: - Ensures 'remote_folder_path' exists under 'base_id' (creating folders as needed) - Uploads 'local_path' to the resulting folder via [syn_upload_file()]

Usage

```
syn_upload_file_path(
  local_path,
  base_id,
  remote_folder_path,
  name = NULL,
  contentType = NULL,
  overwrite = FALSE,
  description = NULL,
  annotations = NULL,
  token = NULL,
  verbose = FALSE,
  dry_run = FALSE,
  min_interval_s = 1.5
)
```

Arguments

local_path	Local file path.
base_id	Base container synId (Project or Folder).
remote_folder_path	Remote folder path relative to 'base_id', e.g. "a/b/c".
name	Optional FileEntity name override.
contentType	Optional MIME type override.
overwrite	Logical; overwrite existing FileEntity with same name.
description	Optional description.
annotations	Optional named list for annotations2.
token	Optional token (PAT).
verbose	Logical.
dry_run	Logical.
min_interval_s	Brief period in seconds to pause if using function in a loop uploading multiple times, otherwise will run into issues.

Value

"syn_upload_result" (see [syn_upload_file()]).

Examples

```
## Not run:
res <- syn_upload_file_path(
  local_path = "C:/data/example.csv",
  base_id = "synProjectId",
  remote_folder_path = "zz_smoketests/synapse_rw/test2",
  overwrite = TRUE,
  annotations = list(stage = "smoke", kind = "csv"),
  verbose = TRUE
)
res$file_entity_id

## End(Not run)
```

timed	<i>timed: simple script timer</i>
-------	-----------------------------------

Description

Start/stop timers and print start/finish timestamps plus elapsed time. Supports: - a single unnamed script timer (default), and - optional labeled timers (multiple independent timers).

Usage

timed(action, label = NULL, round = 2, ret = FALSE)

Arguments

action	Character(1). One of "start", "end", or "stop". ("stop" is treated as "end".)
label	Character(1) or NULL. Optional label to manage independent timers. If NULL (default), an unnamed script timer is used.
round	Integer(1). Decimal places for rounding elapsed time. Default 2.
ret	Logical(1). If TRUE, returns the start time (on "start") or the elapsed difftime (on "end"/"stop"). Default FALSE.

Value

Invisibly NULL by default. If 'ret = TRUE', returns a POSIXct start time (on "start") or a difftime elapsed duration (on "end"/"stop").

Examples

```
timed("start"); Sys.sleep(0.2); timed("end")
timed("start", label = "download"); Sys.sleep(0.1); timed("stop", label = "download")
```

to_camel_case	<i>Convert strings to lowerCamelCase</i>
---------------	--

Description

Converts arbitrary strings to lowerCamelCase by:

1. replacing non-alphanumeric characters with spaces
2. splitting on whitespace
3. lowercasing the first token
4. TitleCasing subsequent tokens
5. concatenating tokens

Usage

to_camel_case(string)

Arguments

string Character vector.

Value

Character vector of the same length, converted to lowerCamelCase.

Examples

```
to_camel_case(c("hello world", "some_value", " already camel "))
```

trim_all_cols	<i>Trim whitespace from all character columns</i>
---------------	---

Description

Applies ‘trimws()’ to all character columns in a data.frame/data.table.

Usage

```
trim_all_cols(data)
```

Arguments

data A data.frame or data.table.

Value

The trimmed object (copy).

Examples

```
d <- data.frame(a = c(" x", "y "), b = 1:2, stringsAsFactors = FALSE)
trim_all_cols(d)
```

Index

[cbind_list](#), [3](#)
[check_ram](#), [3](#)
[collapse](#), [4](#)

[dp_check_units](#), [5](#)
[dp_detect](#), [6](#)
[dp_open](#), [6](#)
[dp_print_detect](#), [7](#)
[dp_read](#), [8](#)
[dp_read_dataset_meta](#), [8](#)
[dp_read_meta](#), [9](#)
[dp_scale_decode](#), [10](#)
[dp_scale_encode](#), [11](#)
[dp_set_scale](#), [12](#)
[dp_set_units](#), [12](#)
[dp_spec](#), [13](#)
[dp_spec_default](#), [14](#)
[dp_tag_units](#), [15](#)
[dp_write](#), [16](#)
[dp_write_dataset](#), [17](#)
[dp_write_dataset_meta](#), [18](#)
[drop_null](#), [19](#)

[env_close](#), [20](#)
[env_start](#), [21](#)

[filter_cols](#), [22](#)
[flatten_list_columns](#), [23](#)

[gp_geoparquet_attach_geo_to_table](#), [23](#)
[gp_geoparquet_build_geo_meta](#), [24](#)
[gp_geoparquet_geo_json](#), [25](#)
[gp_geoparquet_infer_from_sf](#), [25](#)
[gp_geoparquet_is_geoparquet](#), [26](#)
[gp_geoparquet_read_geo](#), [27](#)
[gp_geoparquet_write_parquet](#), [27](#)

[mavtime](#), [28](#)
[merge_dt_list](#), [29](#)

[null_coalesce](#), [29](#)

[print.dp_spec](#), [30](#)

[rep_var](#), [30](#)

[require_ram](#), [31](#)
[round_cols](#), [31](#)

[s3_create_dir](#), [32](#)
[s3_download_file](#), [33](#)
[s3_list_objects](#), [33](#)
[s3_upload_file](#), [35](#)
[sanitize_value](#), [36](#)
[stop_cluster](#), [37](#)
[syn_create_file_entity](#), [37](#)
[syn_create_folder](#), [38](#)
[syn_download_file](#), [39](#)
[syn_download_file_path](#), [40](#)
[syn_ensure_folder_path](#), [41](#)
[syn_get_annotations](#), [41](#)
[syn_get_child_by_name](#), [42](#)
[syn_get_entity](#), [43](#)
[syn_lookup_child_id](#), [43](#)
[syn_request](#), [44](#)
[syn_resolve_entity_by_path](#), [45](#)
[syn_resolve_token](#), [46](#)
[syn_set_annotations](#), [47](#)
[syn_try_get_child_by_name](#), [47](#)
[syn_update_file_entity](#), [48](#)
[syn_upload_file](#), [49](#)
[syn_upload_file_path](#), [50](#)

[timed](#), [52](#)
[to_camel_case](#), [52](#)
[trim_all_cols](#), [53](#)